CS 3563: Introduction to DBMS II

Homework 5: Transaction Management

Name: Pathri Vidya Praveen
Roll No.: CS24BTECH11047
Due Date: 26 March 2026, 11:59 PM IST

Question 1

Lock Compatibility Matrix

To construct the compatibility matrix, we examine whether two operations on the same account can be executed concurrently without affecting the final balance. Two operations are considered compatible if executing them in any order yields the same result (i.e., they are commutative).

Interpretation of Operations

- $R(a, v)$: Reads the current balance of account a.
- $S(a, v)$: Sets the balance of account a to a specified value.
- $D(a, v)$: Deposits an amount v into account a (increases the balance).
- $W(a, v)$: Withdraws an amount v from account a (decreases the balance).

Operations R and S behave like classical read and write actions, while D and W modify the balance arithmetically.

Lock Compatibility Matrix

	$R(a,v)$	$S(a,v)$	$D(a,v)$	$W(a,v)$
$R(a,v)$	Y	N	N	N
$S(a,v)$	N	N	N	N
$D(a,v)$	N	N	Y	Y
$W(a,v)$	N	N	Y	Y

Justification

1. **R with R: Compatible**
   Multiple reads do not modify the balance and therefore can occur simultaneously without interference.

2. **R with S, D, or W: Incompatible**
   Operations S, D, and W change the account balance. A concurrent read may observe inconsistent or intermediate values, so these combinations are not allowed.

3. **S with any operation: Incompatible**
   $S(a, v)$ overwrites the balance with a new value. Any concurrent read or modification could either see an inconsistent value or be lost due to the overwrite, so S conflicts with all other operations.

4. D **with** D **and** W **with** W**: Compatible**
   Deposits and withdrawals are arithmetic updates. When applied to the same initial balance, their final result depends only on the total net change, not on the order of execution. Hence, concurrent deposits or concurrent withdrawals do not interfere with each other.

5. D **with** W**: Compatible**
   A deposit followed by a withdrawal (or vice versa) produces the same final balance as long as both operations are applied. Therefore, these operations commute and can proceed concurrently.

6. D **or** W **with** R **or** S**: Incompatible**
   Since D and W modify the balance, they conflict with reads (which may observe inconsistent values) and with set operations (which overwrite the balance).

Operations D (deposit) and W (withdraw) are compatible with each other because they are commutative arithmetic updates. However, any operation that overwrites or reads the balance (S or R) conflicts with modifications. Thus, the matrix reflects compatibility based on whether concurrent execution preserves the correctness of the final account balance.

Question 2

Conflict Serializability and 2PL Analysis

The given schedule is:

$S: R_2(d),\ R_1(a),\ W_1(a),\ R_3(b),\ R_3(c),\ R_2(a),\ W_2(a),\ R_2(c),\ W_3(c),\ R_1(d),\ R_3(d),\ W_1(d)$

1. Conflict-Serializability

Two operations conflict if they belong to different transactions, access the same data item, and at least one is a write.

Conflicts by data item:

- **Item** a**:**
  - $W_1(a)$ precedes $R_2(a)$ and $W_2(a)$
  - Edge: $T_1 \to T_2$
- **Item** c**:**
  - $R_2(c)$ precedes $W_3(c)$
  - Edge: $T_2 \to T_3$
- **Item** d**:**
  - $R_2(d)$ precedes $W_1(d)$
  - Edge: $T_2 \to T_1$
  - $R_3(d)$ precedes $W_1(d)$

– Edge: $T_3 \to T_1$

The precedence graph therefore contains edges:

$$T_1 \to T_2, \quad T_2 \to T_3, \quad T_2 \to T_1, \quad T_3 \to T_1$$

Since the graph contains the cycle

$$T_1 \to T_2 \to T_1,$$

the schedule is **not conflict-serializable**.

2. Can 2PL Produce This Schedule?

No. The schedule cannot be generated by the Two-Phase Locking (2PL) protocol.

Under 2PL, a transaction cannot acquire any new locks after it has released a lock (i.e., after entering the shrinking phase).

Consider transaction T_1:

- T_1 performs $W_1(a)$, requiring an exclusive lock on item a.
- Later, T_2 accesses item a ($R_2(a)$), which implies that T_1 must have released its lock on a.
- After this point, T_1 later performs $R_1(d)$ and $W_1(d)$, which would require acquiring a new lock on item d.

Since T_1 would need to obtain a new lock after releasing one, this violates the two-phase locking rule.

Conclusion:

- The schedule is not conflict-serializable.
- Therefore, it cannot be produced by a 2PL scheduler.

Question 3

Response:

The vendor's guarantee eliminates system failures such as crashes, but transactions may still abort due to internal reasons. Therefore, the need for a recovery manager depends on how updates are propagated to disk.

Transaction Failures

Even on a permanently operational system, transactions can abort due to:

- Concurrency control decisions (e.g., deadlock resolution)
- Logical conditions detected during execution
- Explicit user actions such as issuing a `ROLLBACK`

These aborts may leave partial effects that must be handled appropriately.

Immediate Update Scheme

In an immediate update system, database pages may be written to disk before the transaction commits.

- Uncommitted updates can reach disk.
- If a transaction aborts, these changes must be undone.

Hence, a recovery mechanism is required to restore the database to its previous consistent state.

Deferred Update Scheme

In a deferred update system, updates are applied to the database only after the transaction successfully commits.

- Uncommitted changes never reach disk.
- Aborted transactions therefore leave no persistent effects.

Under the assumption that the system never fails, there is no need for crash recovery or disk-based undo. Aborted transactions can simply be discarded.

Conclusion

A recovery manager is necessary for immediate update systems because aborted transactions may have modified the database on disk. However, for deferred update systems operating on a failure-free platform, the recovery manager is not required since uncommitted changes never become permanent.

Question 4

Answer:

Yes, the system provides recoverability. The protocol follows the Write-Ahead Logging (WAL) principle, since each log record is flushed to stable storage before the corresponding data page is written to disk. This guarantees that the necessary information to reverse any update is safely recorded prior to modifying the database.

Because the system allows data pages to be written before the transaction commits (immediate update), the disk may contain changes from transactions that later abort. Therefore, recovery must perform **UNDO** operations to remove the effects of such incomplete transactions. Using the before-images stored in the log, the recovery manager restores the database to its previous consistent state.

At commit time, however, all modified pages of a transaction are forced to disk before the commit record is written. Consequently, once a transaction is recorded as committed, its updates are already reflected on disk. There is no need to reapply these changes during recovery.

Thus, recovery requires:

- **UNDO of aborted (uncommitted) transactions**

- **No REDO of committed transactions**

Conclusion: The system implements an **UNDO / NO-REDO** recovery protocol, which is recoverable but incurs additional disk I/O at commit time due to forcing updated pages to disk.